

Safe integral comparisons

I. Table of Contents

- [I\).....Table of Contents](#)
- [II\).....Motivation](#)
- [III\).....Proposal](#)
- [IV\).....Examples](#)
- [V\).....Possible implementation](#)
- [VI\).....Effects on Existing Code](#)
- [VII\).....Design Decisions](#)
- [VIII\)....Further Considerations](#)
- [IX\).....Related Works](#)
- [X\).....Proposed Wording](#)

II. Motivation

Comparing integrals of different types may be a more complex task than expected. Most of the time we expect that a simple

```
if(a < b){  
    // ...  
} else {  
    // ...  
}
```

should work in all cases, but if `a` and `b` are of different types, things are more complicated.

If `a` is a signed type, and `b` unsigned, then, supposing that no integral promotion is taking place, `a` is converted to the unsigned type. If `a` holds a number less than zero, then the result may be unexpected, since the expression `a < b` would evaluate to false, even though a strictly negative number is always lower than a positive one. The reason of this behavior is that unsigned types have modular arithmetic, but most of the time, for example when working with containers, when mixing signed and unsigned types, we want to have integer arithmetic. Also, converting integrals between different types can be challenging. For simplicity, most of the time we assume that values are in range, and write

```
a = static_cast<decltype(a)>(b);
```

If we want to write a safe conversion, we need to check if `b` has a value between `std::numeric_limits<decltype(a)>::min()` and `std::numeric_limits<decltype(a)>::max()`. We also need to pay attention that no implicit conversion (for example between unsigned and signed types) invalidates our comparison.

Comparing and converting numbers, even of different numeric types, should be a trivial task. Unfortunately it is not, and because of implicit conversions we may write, without noticing it, unsafe code.

Most compilers are able to provide diagnostics and generate warnings when comparing values of different types, or when doing a narrowing conversion.

Developers are tempted to assume that values will mostly be in range and write a simple, but possibly wrong, cast in order to silence the warning, or not to turn on the corresponding compiler warning at all.

III. Proposal

This paper proposes to add a set of `constexpr` and `noexcept` functions for converting and comparing integrals of different signedness, except for `bool` and character types.

- Two functions to compare if two variables represent the same value or not

```
template <class T, class U>  
constexpr bool std::cmp_equal(T t, U u) noexcept;  
  
template <class T, class U>  
constexpr bool std::cmp_not_equal(T t, U u) noexcept;
```

- A set of functions that can be used to determine the relative order of two values

```
template <class T, class U>
constexpr bool std::cmp_less(T t, U u) noexcept;

template <class T, class U>
constexpr bool std::cmp_greater(T t, U u) noexcept;

template <class T, class U>
constexpr bool std::cmp_less_equal(T t, U u) noexcept;

template <class T, class U>
constexpr bool std::cmp_greater_equal(T t, U u) noexcept;
```

- One function to determine if a specific value is inside the range of possible values of another type (i.e. if we can convert the value to the other type safely)

```
template <class R, class T>
constexpr bool in_range(T t) noexcept;
```

IV. Examples

Examples without current proposal

Comparing an unsigned int with an int:

```
int a = ...
unsigned int b = ...
// add static_cast to avoid compiler warnings since we are doing a "safe" comparison
if(a < 0 || static_cast<unsigned int>(a) < b){
    // do X
} else {
    // do Y
}
```

Comparing a uint32_t with an int16_t:

```
int32_t a = ...
uint16_t b = ...
// add static_cast to avoid compiler warnings since we are doing a "safe" comparison
if(a < static_cast<int32_t>(b)){
    // do X
} else {
    // do Y
}
```

Comparing an int with an intptr_t:

```
int a = ...
intptr_t b = ...
if(???){ // no idea how to do it in one readable line without some assumption about int and intptr_t
    // do X
} else {
    // do Y
}
```

Example with current proposal

Comparing one integral type A with another integral type B (both non bool or character type):

```
A a = ...
B b = ...
// no need for any cast since std::cmp_less is taking care of everything
if(std::cmp_less(a,b)){
    // do X
} else {
    // do Y
}
```

V. Possible implementation

A possible implementation can be found on [github](#). The only dependencies are the `std::numeric_limits` function from the `limits` header, some traits from the `type_traits` header and a standard conforming C++11 compiler.

VI. Effects on Existing Code

Since the proposed functions are not defined in any standard header, the meaning of no existing code will be changed.

VII. Design Decisions

This proposal addresses how to compare *numerical values* of different types (aka standard integer types and extended integer types) in a safe and simple way. It makes little sense to compare `true`, `false`, `'a'` and other characters to numbers, since they represent different logical entities. The encoding of characters is also not specified, therefore the possible valid comparison `'a' == 97` might yield different results depending on the locale, compiler or platform.

Providing an overload for `char` might not reduce confusion, for example:

```
int32_t a = ...
char c = -1;
cmp_less(c, 0) // true if char is signed, false if char is unsigned.
```

If the user has to choose between `signed char` or `unsigned char`, the behaviour will always be consistent. Using `char` for storing a number is a valid use case (the language permits it), but the types `signed char` and `unsigned char` should be preferred since those are standard integer types and have the same size.

I would also recommend not to provide overloads for `bool` and the character types because it is easier to add them later if needed, whereas removing them might be more difficult since it would be a breaking change.

If the LEWG would like to include `char`, I think it would be better to provide an overload for every character type for consistency.

VIII. Further Considerations

I've heard rumors that it might be possible that the current `operator<` et al. could get deprecated and maybe changed someday to behave like the functions proposed in this proposal.

I would like to add some considerations:

- Doing the right thing might be less efficient than doing the wrong thing. Changing how `operator<` works on integral types might make it less efficient, it may require extra instructions, even an extra branch instruction. Performance is mostly irrelevant if we need to choose between the right result and a possibly wrong result. Compilers are able to detect when comparing numbers of different types and they'll very probably be able to do so in the future even if `operator<` changes meaning. If a developer wants better efficiency, they should use the same type to avoid conversions.
- Even today, comparing numbers might require more instructions and branches than expected on some targets.
- Because of optimizations and branch prediction, the `cmp_less` function might be as efficient as the current `operator<`.
- There are some use cases where, today, we have a warning as a side-effect that shows the user that the code might be wrong, but by changing `operator<` it will still be wrong and we will not have the warning anymore:

```
for(auto i = 0; i < container.size(); ++i){/**/}.
```

The code is wrong with all standard containers because the condition may never be met and there is a possible overflow. Since we are comparing, we get a warning because of `operator<`. The problem is that in this case it's not the comparison that is wrong, but the whole expression (it could also be that `size` returns a signed type but with a bigger range). As stated above, the warning caused by `operator<` is just a fortunate side-effect. I do not know if compilers in the future will be able to warn about those and more complex expressions.

- Changing how comparison operators works, but not the assignment operator, will lead to inconsistencies, and can create to subtle bugs:

```
unsigned int u = std::numeric_limits<int>::max();
int s = -1;
assert(s!=u); // supposing that operator!= compares between signed and unsigned without modulo behaviour
u = s;
assert(s == u); // expected to pass, but will fail
```

whereas simply deprecating the comparison would enhance the possibilities to spot the error.

IX. Related Works

In 2016, Robert Ramey did a much bigger proposal (see [p0228r0](#)) regarding safe integer types. He also used functions similar to

those proposed in this paper for implementing his classes and operators, so an alternative implementation can be found on his [github repository](#). This proposal addresses a smaller problem, namely comparing integral values, and is therefore much smaller. The functions provided can be also used for creating safe integer types.

Another work, by Herb Sutter (see [p0515r3](#)), is about a new comparison operator (`<=>`). In its current state the operator `<=>` will not compare different integral types, but in a previous revision as far as I've understood, the proposal stated that operator `<=>` should compare different integral types without modulo behaviour making part of this proposal obsolete.

X. Proposed Wording

The following changes are relative to [n4659](#).

In “[version.syn]” add the feature test macro `__cpp_lib_cmp_equal` // also defined in `<utility>`.

In “[utility.syn]”, insert before // [intseq], Compile-time integer sequences declarations:

```
// safe integral comparisons
template <class T, class U>
constexpr bool cmp_equal(T t, U u) noexcept;
template <class T, class U>
constexpr bool cmp_not_equal(T t, U u) noexcept;

template <class T, class U>
constexpr bool cmp_less(T t, U u) noexcept;
template <class T, class U>
constexpr bool cmp_greater(T t, U u) noexcept;
template <class T, class U>
constexpr bool cmp_less_equal(T t, U u) noexcept;
template <class T, class U>
constexpr bool cmp_greater_equal(T t, U u) noexcept;

template <class R, class T>
constexpr bool in_range(T t) noexcept;
```

Append a new Section “safe integral comparisons” in “[utility]”, with the following content:

- ```
template <class T, class U>
constexpr bool cmp_equal(T t, U u) noexcept;
```

Mandates: Both `T` and `U` are standard integer types or extended integer types ([basic.fundamental]).

Effects: Equivalent to:

```
using UT = make_unsigned_t<T>;
using UU = make_unsigned_t<U>;
if constexpr (is_signed_v<T> == is_signed_v<U>)
 return t == u;
else if constexpr (is_signed_v<T>)
 return t < 0 ? false : UT(t) == u;
else
 return u < 0 ? false : t == UU(u);
```

- ```
template <class T, class U>
constexpr bool cmp_not_equal(T t, U u) noexcept;
```

Effects:

Equivalent to: `return !cmp_equal(t,u);`

- ```
template <class T, class U>
constexpr bool cmp_less(T t, U u) noexcept;
```

Mandates: Both `T` and `U` are standard integer types or extended integer types ([basic.fundamental]).

Effects: Equivalent to:

```
using UT = make_unsigned_t<T>;
using UU = make_unsigned_t<U>;
if constexpr (is_signed_v<T> == is_signed_v<U>)
 return t < u;
else if constexpr (is_signed_v<T>)
 return t < 0 ? true : UT(t) < u;
else
 return u < 0 ? false : t < UU(u);
```

- ```
template <class T, class U>
constexpr bool cmp_greater(T t, U u) noexcept;
```

Effects:

Equivalent to: `return cmp_less(u, t);`

- `template <class T, class U>`
`constexpr bool cmp_less_equal(T t, U u) noexcept;`

Effects:

Equivalent to: `return !cmp_greater(t, u);`

- `template <class T, class U>`
`constexpr bool cmp_greater_equal(T t, U u) noexcept;`

Effects:

Equivalent to: `return !cmp_less(t, u);`

- `template <class R, class T>`
`constexpr bool in_range(T t) noexcept;`

Mandates: Both `T` and `R` are standard integer types or extended integer types ([basic.fundamental]).

Effects:

Equivalent to: `return cmp_greater_equal(t, numeric_limits<R>::min()) && cmp_less_equal(t, numeric_limits<R>::max());`

[Note: These function templates cannot be used to compare `byte`, `char`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, and `bool`. --end note]